Artificial Intelligence Technology and Application

Machine Learning Lab Guide

Student Version

HUAWEI

Huawei Technologies CO., LTD.

Contents

1 Feature Engineering on Banks' Private Credit Data 1
1.1 Introduction 1
1.1.1 About This Lab 1
1.1.2 Objectives 1
1.1.3 Case Background 1
1.2 Data Preprocessing 2
1.2.1 Background 2
1.2.2 Procedure 2
1.3 Feature Selection 5
1.3.1 Background 5
1.3.2 Feature Selection Methods 5
1.3.3 Filter 5
1.3.4 Wrapper 8
1.3.5 Embedded 8
1.3.6 Variable Removal 10
1.4 Feature Construction 10
1.4.1 Background 10
1.4.2 Polynomial Feature Construction 10

1 Feature Engineering on Banks' Private Credit Data

1.1 Introduction

1.1.1 About This Lab

Feature engineering is a process of extracting features from raw data. Data and features determine the upper limit of machine learning, while models and algorithms help continuously approaching this upper limit. Feature engineering and construction aim to enable extracted features to represent the essential characteristics of data to the greatest extent, so that a model constructed based on these features has a good prediction effect on unknown datasets.

1.1.2 Objectives

Upon completion of this task, you will be able to:

- Master the Python-based feature selection method.
- Master the Python-based feature extraction method.
- Master the Python-based feature construction method.

1.1.3 Case Background

With the development of online financial services, bank H plans to evaluate customer risks by using online approval to reduce labor costs and improve approval efficiency. Online approval requires a more strict and accurate risk control model to control corporate financial risks. Therefore, algorithm engineer A needs to complete feature engineering and construction of a credit risk model before constructing the model based on historical customer credit data. Engineer A needs to complete the following operations:

- Data preprocessing
- Feature selection
- Feature construction

1.2 Data Preprocessing

1.2.1 Background

The raw data collected by the back-end server of the bank may have problems such as missing values, garbled characters, redundant fields, and inconsistent data formats. To improve the data quality, engineer A needs to cleanse the data first.

1.2.2 Procedure

1.2.2.1 Importing Data

Code:

```
import pandas as pd
```

Output:

Cust_No	Target	Nation	Birth_Place	Gender	Age	Marriage_State	Highest Education	House_State	Work_Years	Title	...	ZX_Max_Overdue_Account	ZX_Link_Max_C
2	0	1.0	330621	1	55	40.0	71.0	1.0	0	9.0	...	1	
4	0	1.0	330621	0	40	99.0	90.0	1.0	0	9.0	...	0	
6	0	1.0	330621	1	45	20.0	71.0	1.0	0	9.0	...	1	
7	0	1.0	330421	0	32	20.0	21.0	1.0	0	9.0	...	1	
8	0	1.0	330621	0	46	20.0	71.0	1.0	0	9.0	...	0	

5 rows × 31 columns

1.2.2.2 Processing Missing Values

Step 1 View the missing values.

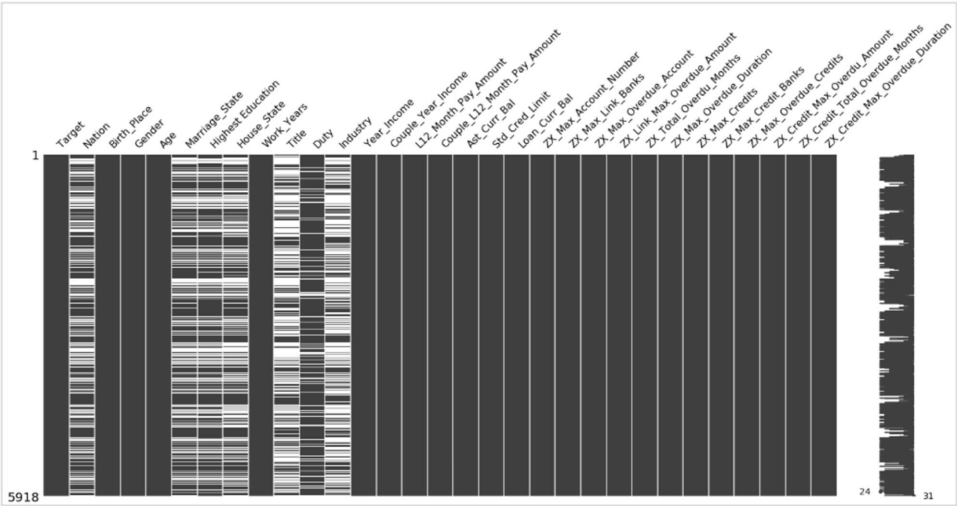
The missing values in the data may be caused by machine faults, manual input errors, or service attributes. The method for processing the missing values varies with the cause.

missingno is a tool for visualizing missing values. You can run the following command to view the missing-value distribution in the data:

Code:

```
import missingno # Import the missingno package.
```

Output:



As shown in the figure above, the **Nation**, **Marriage_State**, **Highest Education**, **House_State**, **Industry**, **Title**, and **Duty** fields contain a large number of missing values. In Pandas, **isnull()** can determine the missing values in data, and **isnull().sum()** can count the number of missing values and further check the rates of the missing values in the fields.

Code:

Output:

	index	missing_rate
9	Title	0.603075
11	Industry	0.594120
7	House_State	0.424806
1	Nation	0.356877
5	Marriage_State	0.334404
6	Highest Education	0.325786
10	Duty	0.126563
25	ZX_Max_Credits	0.000000
21	ZX_Max_Overdue_Account	0.000000
22	ZX_Link_Max_Overdue_Amount	0.000000
23	ZX_Total_Overdu_Months	0.000000
24	ZX_Max_Overdue_Duration	0.000000
0	Target	0.000000
19	ZX_Max_Account_Number	0.000000
26	ZX_Max_Credit_Banks	0.000000

Step 2 Fill the missing values with the mode.

Pandas provides **fillna()** to fill the missing values, and **mode()** to fill the missing values with the mode. You need to construct a for loop to process multiple fields that contain missing values and fill the missing values with the mode.

Define the list of fields with missing values.

Use the for loop to process the missing values in the multiple fields.

After the processing is complete, check the missing rate of each field.

	index	missing_rate
0	Target	0.0
16	Ast_Curr_Bal	0.0
29	ZX_Credit_Total_Overdue_Months	0.0
28	ZX_Credit_Max_Overdu_Amount	0.0
27	ZX_Max_Overdue_Credits	0.0
26	ZX_Max_Credit_Banks	0.0
25	ZX_Max_Credits	0.0
24	ZX_Max_Overdue_Duration	0.0
23	ZX_Total_Overdu_Months	0.0
22	ZX_Link_Max_Overdue_Amount	0.0
21	ZX_Max_Overdue_Account	0.0
20	ZX_Max_Link_Banks	0.0
19	ZX_Max_Account_Number	0.0
18	Loan_Curr_Bal	0.0
17	Std_Cred_Limit	0.0

The following are methods for handling missing values:

1. Direct deletion: deletes the non-important service fields with the missing rate greater than 80%.
2. Data filling: fills the missing values with the determined values, statistical indicator values, and algorithm-based missing value predictions.
3. Separate processing: processes samples with missing values as one category.

----End

1.3 Feature Selection

1.3.1 Background

If excessive features are obtained after data preprocessing, the model may be unstable and have poor generalization capability. As a result, the computing complexity increases exponentially. Therefore, engineer A needs to preliminarily filter out features that are not important to the prediction result.

1.3.2 Feature Selection Methods

The following are methods for feature selection:

- Filter: filters features based on the statistical indicators for the feature vectors and those common to the feature vectors and the target variables.
- Wrapper: attempts to use different feature subsets for modeling and use the model precision as an evaluation indicator for the feature subsets.
- Embedded: evaluates feature weights during model training and scores the importance of the features.

1.3.3 Filter

Step 1 Analyze the crosstab.

Apply the **crosstab()** method to draw a crosstab by using the variable **House_State** and the target variable **Target** as an example.

Target	0	1	All
House_State			
1.0	0.980996	0.019004	1.0
2.0	0.954545	0.045455	1.0
3.0	0.941176	0.058824	1.0
4.0	1.000000	0.000000	1.0
5.0	0.980392	0.019608	1.0
6.0	1.000000	0.000000	1.0
7.0	0.857143	0.142857	1.0
8.0	1.000000	0.000000	1.0
All	0.980399	0.019601	1.0

In the output, the default rate is 0.019 when **House_State** is set to **1**, and is 0.045 when **House_State** is set to **2**. If the default rates are considered the same, the variable **House_State** does not affect the default prediction.

The crosstab analysis can only be used for preliminary judgment and analysis. The chi-square test is further needed to determine whether the numerical difference has statistical significance.

Step 2 Perform the chi-square test.

Separate independent variables and dependent variables from the raw data, and select categorical variables from the independent variables.

The **Target** field is a target variable and is assigned to **y**. The column with the target variable removed is assigned to **X** as an independent variable. **X_category** indicates a categorical variable.

Import the chi-square test package **chi2** of sklearn.feature_selection and use **chi2()** to calculate the chi-square values of each categorical variable and target variable.

```
[('Work_Years', 30037.98992988671),
 ('Birth_Place', 2337.714562647344),
 ('Marriage_State', 42.7575821276435),
 ('Duty', 31.147387755026166),
 ('Industry', 25.452013582101742),
 ('Nation', 5.256723621174332),
 ('Gender', 2.309012664555949),
 ('Title', 0.8774406202190365),
 ('House_State', 0.3184384017473372),
 ('Highest Education', 0.08532013488252715)]
```

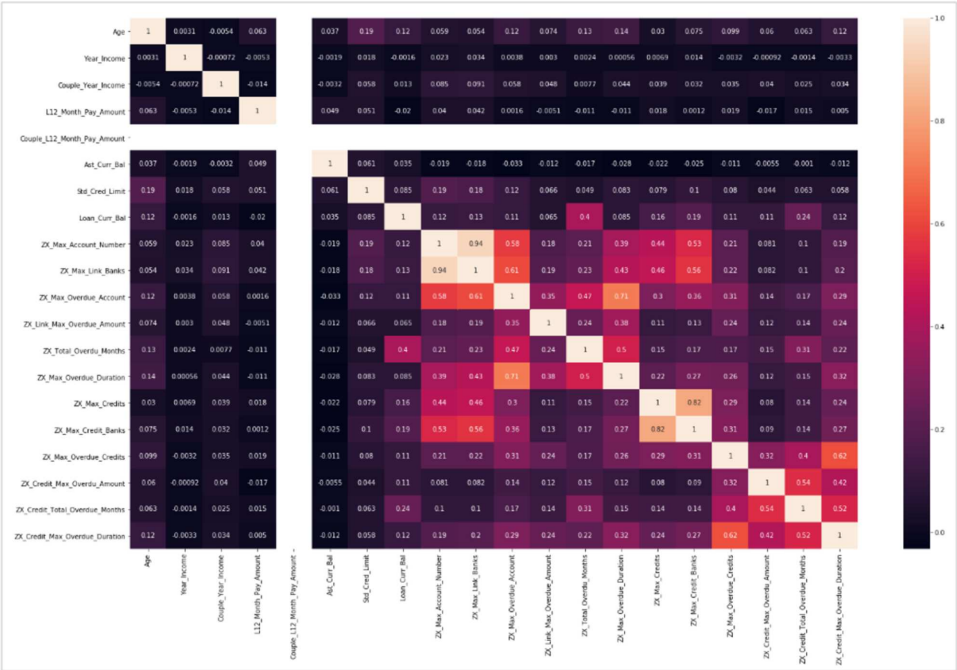
Step 3 Test the continuous variable correlation.

If two continuous independent variables are highly correlated, delete one of the two independent variables or extract common information from the two independent variables.

The **method** parameter indicates the method for calculating the correlation coefficient. The options are as follows:

- **pearson**: Pearson correlation coefficient.
- **kendall**: correlation coefficient for unordered categorical variables.
- **spearman**: Spearman correlation coefficient, which is mainly used for correlation analysis of non-linearly and non-normally distributed data.

```
import matplotlib.pyplot as plt
import seaborn as sns
```



Calculate the correlation coefficient between continuous independent variables and select the combination of independent variables whose correlation coefficient is greater than 0.8.

----End

1.3.4 Wrapper

In the wrapper selection method, different feature subsets are used for modeling, the model precision is used as the evaluation indicator for the feature subsets, and a base model is selected to perform multi-round training. After each round of training, features of some weight coefficients are removed, and then the next round of training is performed based on the new feature set. The **RFE()** method of the feature_selection submodule in sklearn is invoked. The logistic regression model **LogisticRegressio()** is used as the base model to be invoked, and parameter will be transferred into this model.

Wrapper:

estimator: basic training model, which is a logistic regression model in this example.

n_features_to_select: indicates the number of retained features.

fit(X,y): invokes and trains a model.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
```

Output:

```
20
[ True True False True True True False True True True True False
False True False True True False True True True True True True
True False False False True False]
[ 1 1 9 1 1 1 10 1 1 1 1 6 3 1 11 1 1 8 1 1 1 1 1 1
1 5 4 7 1 2]
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
intercept_scaling=1, max_iter=100, multi_class='warn',
n_jobs=None, penalty='l2', random_state=None, solver='warn',
tol=0.0001, verbose=0, warm_start=False)
```

The return value of the **RFE()** method is output, which can be any of the following:

- **n_features_**: number of selected features, that is, the value of the **n_features_to_select** parameter transferred into the **RFE()** method.
- **support_**: indicates that the selected features are displayed at their corresponding positions. **True** indicates that the feature is retained, and **False** indicates that the feature is removed.
- **ranking_**: indicates the feature ranking. **ranking_[i]** corresponds to the ranking of the ith feature. The value **1** indicates the optimal feature. The selected features are the 20 feature corresponding to the value **1**, namely, the optimal features.
- **estimator_**: returns the parameters of the base model.

1.3.5 Embedded

The embedded method uses a machine learning model for training to obtain weight coefficients of features, and selects features in descending order of the weight coefficients.

Common embedded methods are based on either of the following:

- Linear model and regularization
- Feature selection of a tree model

In the tree model, the importance of a feature is determined by the depth of the leaf node. In this experiment, the random forest is used to calculate the importance of a feature.

The random forest classification method in the sklearn.ensemble submodule is invoked to train the model by using the **fit(X,y)** method.

```
from sklearn.ensemble import RandomForestClassifier
```

After the model training is complete, the weight evaluation value of each feature is printed.

```

```

Output:

```
[(0.1315, 'Ast_Curr_Bal'),
(0.1286, 'Age'),
(0.0862, 'Year_Income'),
(0.0649, 'Std_Cred_Limit'),
(0.043, 'ZX_Max_Account_Number'),
(0.0427, 'Highest Education'),
(0.0416, 'ZX_Link_Max_Overdue_Amount'),
(0.0374, 'ZX_Max_Link_Banks'),
(0.0355, 'Industry'),
(0.0354, 'ZX_Max_Overdue_Duration'),
(0.0311, 'ZX_Total_Overdu_Months'),
(0.0305, 'Marriage_State'),
(0.0305, 'Duty'),
(0.0292, 'Couple_Year_Income'),
(0.0279, 'ZX_Credit_Max_Overdu_Amount'),
(0.0246, 'ZX_Max_Overdue_Account'),
(0.0241, 'ZX_Max_Credit_Banks'),
(0.0221, 'ZX_Max_Credits'),
(0.0205, 'Birth_Place'),
(0.0195, 'Loan_Curr_Bal'),
(0.0173, 'L12_Month_Pay_Amount'),
(0.015, 'ZX_Credit_Max_Overdue_Duration'),
(0.013, 'Title'),
(0.0097, 'ZX_Credit_Total_Overdue_Months'),
(0.0096, 'Nation'),
(0.0084, 'Gender'),
(0.0079, 'Work_Years'),
(0.0064, 'ZX_Max_Overdue_Credits'),
(0.0059, 'House_State'),
(0.0, 'Couple_L12_Month_Pay_Amount')]
```

1.3.6 Variable Removal

Based on the results of the preceding three variable selection methods, the variables that have a small model impact weight are removed.

Cust_No	Target	Nation	Birth_Place	Age	Marriage_State	Highest Education	Work_Years	Title	Duty	Industry	...	Ast_Curr_Bal	Std_Cred_Limit	ZX_Max_Account_I
2	0	1.0	330621	55	40.0	71.0	0	9.0	9.0	52.0	...	523.32	10000	
4	0	1.0	330621	40	99.0	90.0	0	9.0	2.0	51.0	...	2305.25	20000	
6	0	1.0	330621	45	20.0	71.0	0	9.0	1.0	17.0	...	17299.71	20000	
7	0	1.0	330421	32	20.0	21.0	0	9.0	0.0	83.0	...	122.24	20000	
8	0	1.0	330621	46	20.0	71.0	0	9.0	0.0	51.0	...	91.82	5000	

5 rows × 23 columns

1.4 Feature Construction

1.4.1 Background

Feature selection is used to find the importance of each feature to model construction and remove variables that have little impact on model construction to reduce dimensions. Feature construction is to manually discover some significant model features from raw data. On the one hand, a new variable can be constructed by combining several different features based on service understanding; on the other hand, variables of different time windows can be divided according to a time attribute.

Engineer A has completed the preliminary filtering of features, removed some variables that have little impact on the model, and now attempts to construct some new features to improve the model precision.

1.4.2 Polynomial Feature Construction

Polynomial feature construction is to explore the impact of complex variables on the target variable through product transformation for existing features. The **PolynomialFeatures()** method in the sklearn.preprocessing submodule is used for feature interaction. Variables with higher scores in the model, that is, **Ast_Curr_Bal**, **Age**, **Year_Income**, and **Std_Cred_Limit**, are selected to construct a polynomial feature.

PolynomialFeatures(degree=3): specifies the degree 3 of interaction between variables, that is, constructs a polynomial whose sum of powers of two variables is 3.

```
from sklearn.preprocessing import PolynomialFeatures
# Select fields used to construct the polynomial feature.
```

```
# Invoke the fit() method to construct the polynomial feature.

# Convert data.
```

To check the correlation between the newly generated variable and the target variable, construct a dataset containing the target variable and the newly generated variable first.

```
```

	1	Ast_Curr_Bal	Age	Year_Income	Std_Cred_Limit	Ast_Curr_Bal^2	Ast_Curr_Bal Age	Ast_Curr_Bal Year_Income	Ast_Curr_Bal Std_Cred_Limit	Age^2
0	1.0	523.32	55.0	100000.0	10000.0	2.738638e+05	28782.60	5.233200e+07	5233200.0	3025.0
1	1.0	2305.25	40.0	300000.0	20000.0	5.314178e+06	92210.00	6.915750e+08	46105000.0	1600.0
2	1.0	17299.71	45.0	150000.0	20000.0	2.992800e+08	778486.95	2.594956e+09	345994200.0	2025.0
3	1.0	122.24	32.0	80000.0	20000.0	1.494262e+04	3911.68	9.779200e+06	2444800.0	1024.0
4	1.0	91.82	46.0	50000.0	5000.0	8.430912e+03	4223.72	4.591000e+06	459100.0	2116.0

The **corr()** function is used to calculate the correlation coefficient between the newly generated variable and the target variable.

```
```

Output:

```
Five features with the smallest correlation coefficients:
Age^3 -0.010601
Age^2 -0.009275
Age^2 Std_Cred_Limit -0.008064
Age -0.007356
Age Std_Cred_Limit -0.006834
Name: Target, dtype: float64
Five features with the largest correlation coefficients:
Year_Income^3 -0.001910
Ast_Curr_Bal Age -0.001114
Ast_Curr_Bal 0.002849
Target 1.000000
1 NaN
Name: Target, dtype: float64
```